

Table 3

The entity types and relation types for CONCEPTRULES V1 (simplified/full), CONCEPTRULES V2 (simplified/full), PARARULES, and our PARARULE-Plus.

Dataset	#Entity	#Relation	Shuffled Rules	Depth Tag	Derivable	NAF
CONCEPTRULES V1 (simplified)	385	7	No	No	Yes	Yes
CONCEPTRULES V1 (full)	4048	24	Yes	No	Yes	No
CONCEPTRULES V2 (simplified)	385	7	No	Yes	Yes	Yes
CONCEPTRULES V2 (full)	4048	24	Yes	Yes	Yes	Yes
PARARULES	19	4	No	Yes	Yes	Yes
PARARULE-Plus	71	8	No	Yes	Yes	Yes

network trained on symbolic logic programs. For details about DeepLogic, we refer the reader to the original paper [1]. The main differences in the work presented here are that we adapt the DeepLogic model to learning logic expressed in natural language, and the model operates at the word-level instead of character-level. The main architecture of the iteration framework is the same as DeepLogic (shown in Figure 3).

Word-level embedding. The input representation layer of the network takes a sequence of words concatenated from two sentences $w_0^C, \dots, w_m^C$ and $w_0^S, \dots, w_n^S$ for context and question respectively.

$$h_t = \text{GRU}(\text{GloVe}[\mathbf{I}_{::t}^C + \mathbf{I}_{::t}^S], h_{t-1}) \quad (1)$$

The context $\mathbf{I}^C$ and the question $\mathbf{I}^S$ at time step t are embedded by $\text{GloVe}[\mathbf{I}_{::t}^C + \mathbf{I}_{::t}^S]$, the GloVe [32] word vector representation. GloVe is a set of large-scale pretrained word vectors. We use GloVe instead of the character-level embedding in DeepLogic. In DeepLogic, the logic programs are expressed by symbols using English letters and other characters. However, the logic programs in our settings are expressed in natural language. Representing the programs using word embeddings can better capture the semantic information of natural language. From the other perspective, GloVe uses ratios of co-occurrence probabilities to enlarge or narrow the relationship between words of different or similar meanings [32]. At the same time, Word2Vec uses a local n-gram window to extract information. Furthermore, GloVe achieves better results faster than the other word-level embeddings like Word2Vec on word analogy task [32]. The dimension of the GloVe embedding we used is 1×100 . The dimension of a sentence embedding concatenated with 5 words is 5×100 . The sentence embedding is processed by the gated recurrent unit (GRU) [33]. Hidden state at time t is denoted as h_t . The context vector is $\mathbf{C} \in \mathbb{R}^{R \times L \times d}$, where R is the number of rules, L is the number of words in the rules and d is the dimension of the embedding.

Iteration. Each iteration step computes the new state based on the current state and rules. In our model, gated attention is trained to decide how much of the state information will be reserved from the current state and the previous state. The process is iterated for T steps (T is pre-determined). The initial state is denoted as $s^0 = q$, where q is the question vector and $q \in \mathbb{R}^d$. The i -th rule is denoted as r_i . W and U are the learnable weight matrices. b is the bias vector.

$$w_i^t = [s^t; q; r_i; (s^t - r_i)^2; s^t \odot r_i] \quad (2)$$

$$\alpha_i^t = \sigma(W(Uw_i^t + b) + b) \quad (3)$$

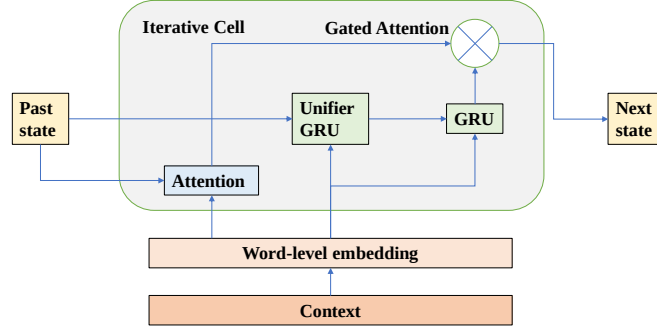


Figure 3: The iterative neural cell with word-level embeddings as input. Questions and contexts are represented as word embeddings, and then attentions are computed to pick up related rules. Gated attention is used to compute the weighted sum of the Unifier GRU outputs. Then the weighted sum updates the state for the next iteration.

At time step t , we compute a feature vector w_i^t using the current state $s^t \in \mathbb{R}^d$, question vector q and a rule r_i . In (2), $[\cdot]$ is a concatenation operator. We use a feed-forward network to compute the final attention vector α_i^t (3), where σ is a sigmoid function. For experimental comparison, softmax is used for our IMA-GloVe-GA and another baseline model IMASM-GloVe to compute the attention vector α_i^t , instead of the sigmoid function.

$$h_{ij}^t = \text{GRU} \left(C_{ij}, h_{i(j-1)}^t \right) \quad (4)$$

$$s^{t+1} = \sum_i^R \alpha_i^t h_{iL}^t \quad (5)$$

We use another recurrent neural network to process the context embedding C_{ij} . The initial hidden state $h_{i0}^t = s^t$, where s^t is the current state. For each rule, the new hidden state h_{ij}^t is computed by (4). In the end, the new state s^{t+1} is computed as a weighted sum of the final hidden states in (5). The Unifier GRU learns the unification between pronouns (variables) and nouns (constants).

Gated attention. Dynamic memory network+ [34] achieved 100% test accuracy by using gated attention on the bAbI deductive reasoning task (Task-15). However, Task-15 is not multi-step deductive reasoning as considered in our experiment. Dynamic memory network with gated attention was shown to perform better than the one with the traditional GRU on bAbI tasks [34]. Although gated attention can perform well on Task-15, it is worth exploring applying an iterative neural network with gated attention to multi-step deductive reasoning. This inspired us to integrate gated attention with DeepLogic which is an iterative neural inference network for multi-step reasoning. We use gated attention to replace dot-product attention, and then update the GRU utilizing the output of gated attention. Gated attention can be seen as one of variety solution from gate mechanism and attention. There are two main existing work related to gated attention. The first one is gate mechanism. The gate mechanism (forget gate in LSTM [35]) is designed to address the issue of forgetting in long and short term memory for recurrent neural network. The other one is gate mechanism and attention that DeepLogic shows that with the

help of GRU and dot-product attention, the model’s performance on the multi-step reasoning tasks can improve a lot from pure GRU without adding dot-product attention. However, in DeepLogic, the dot-product attention is added above on GRU. For each iteration, the attention is used to compute a new state based on the embeddings of the context and question with the previous time stamp state, while the gate in GRU is not specifically trained to keep the useful information with multi-step reasoning. Then we want to use the gated attention to update the gate with attention-enhanced information from context and question in order to improve multi-step reasoning. By replacing the attention vector (4) with gated attention g_i^t (6), the gated attention is used to update the internal state for GRU as in (7).

$$g_i^t = \frac{\exp(W(Uw_i^t + b) + b)}{\sum_{k=1}^R \exp(W(Uw_k^t + b) + b)} \quad (6)$$

$$h_i = g_i^t \circ \tilde{h}_i + (1 - g_i^t) \circ h_{i-1} \quad (7)$$

5. The Datasets

To investigate whether end-to-end neural networks designed for symbolic logic reasoning can be adapted to do multi-step deductive reasoning in natural language, we evaluated the models on PARARULES [3], CONCEPTRULES V1 [6], and CONCEPTRULES V2 [7] datasets that require various depths of reasoning. The main difference between the existing datasets used to evaluate DeepLogic and the multi-step natural language reasoning datasets is that the former are based on logic programs like the examples in Table 1, each predicate is represented by a string of symbols and characters. However, natural language is much more diverse and expressive. Both PARARULES and CONCEPTRULES are synthetically generated datasets. PARARULES also includes examples with sentences paraphrased by humans. These examples are more diverse in language and more challenging for the model. Furthermore, to address the issue of depth imbalance in the current multi-step reasoning datasets, we develop a new dataset called PARARULE-Plus, a large multi-step reasoning dataset over natural language. The dataset includes examples of four reasoning depths, from 2 to 5. There are around 100,000 samples for each depth and nearly 400,000 samples in total. The detailed information about PARARULE-Plus can be found in Table 2 and Appendix.

6. Experiments

We experiment with three variants of DeepLogic. The iterative memory attention (IMA) model is adopted from DeepLogic. IMASM is similar to IMA, except that IMASM uses softmax rather than sigmoid when computing attention scores. We also test three baseline models from the bAbI leaderboard: Long short-term memory (LSTM) [36] (The baseline method on bAbI dataset), dynamic memory networks (DMN) [37] (100% test accuracy on bAbI), and memory attention control networks (MAC) [38] (A classical method using memory network). We use GloVe [32] as the word vector representation for the baseline methods, including IMA-GloVe, IMASM-GloVe, MAC-GloVe, DMN-GloVe, and LSTM-GloVe. Our IMA-GloVe-GA uses gated attention instead of the dot-product attention in the IMA-GloVe model.

We train the models on PARARULES, incorporating all reasoning depths (Depth=0, Depth=1, Depth= 2, Depth=3, Depth=5) and the examples paraphrased by humans (NatLang). We set the random seed to 0. We train the models using the Adam [39] optimiser for 30 epochs. After each epoch, the data is reshuffled, and the rules in the context are reshuffled for each mini-batch. The batch size is 32. The maximum iteration depth is set to 4. The initial learning rate for